École supérieure en Sciences et Technologies de l'Informatique et du Numérique

Module: Cryptographie Avancée
Niveau: 2CS – Cyber-Sécurité

Chapitre 3
La Cryptographie à courbes elliptiques

Présenter par : DR BOUCHOUCHA

Année Universitaire 2024-2025

Introduction

La cryptographie à courbes elliptiques (ECC) repose sur des propriétés mathématiques de courbes elliptiques pour garantir la sécurité des échanges d'informations. Elle offre un niveau de sécurité élevé tout en **utilisant des tailles de clés plus petites** que les autres algorithmes comme RSA, ce qui la rend particulièrement adaptée aux environnements avec des ressources limitées, tels que **les appareils mobiles ou les objets connectés.**

Les courbes elliptiques sont utilisées pour créer des systèmes de clés publiques et privées dans le cadre de protocoles comme **l'échange de clés**, **les signatures numériques**, et **le chiffrement**.

Sa sécurité est basée sur le problème du logarithme discret de la courbe elliptique.

L'ECC (Elliptic Curve Cryptography) est largement utilisée dans les systèmes cryptographiques actuels:

- Dans les systèmes embarqués vue les avantages qu'elle présente en termes de sécurité et d'efficacité comparée à d'autres systèmes cryptographiques asymétriques tel que le RSA.
- Dans les protocoles TLS et SSH pour sécuriser la navigation sur Internet,
- Dans des protocoles comme PACE pour sécuriser les cartes d'identités et les passeports électroniques.

2. Mathématiques des courbes elliptiques

2.1. Définition d'une courbe elliptique

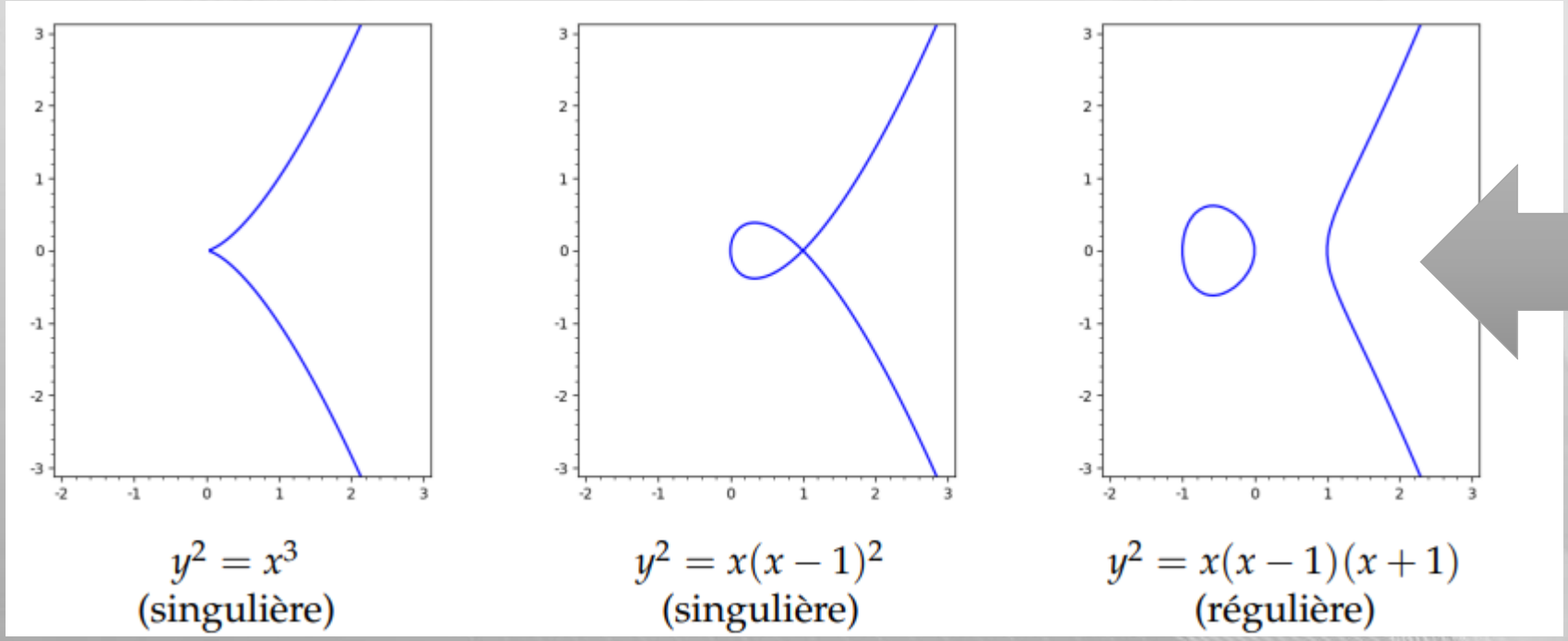
Une courbe elliptique est une courbe algébrique défini par l'ensemble des solutions des couples (x, y) ∈ R × R, qui est défini par l'équation:

$$y^2 = x^3 + ax + b.$$

où a et b sont des coefficients qui satisfont la condition suivante pour garantir que la courbe est lisse et sans singularités :

$$\Delta = \mathbf{4}\boldsymbol{a}^{\mathbf{3}} + \mathbf{27}\,\boldsymbol{b}^{\mathbf{2}} \neq \mathbf{0}$$

Si Le discriminant de la courbe $\Delta = 4a^3 + 27\,b^2 = 0$, on dit que la courbe **est singulière**; c'est un cas particulier que l'on souhaite éviter (en général). ***Sinon, elle est régulière*.**



Courbes elliptiques dans Zp

Les courbes elliptiques sont étudiées dans le cadre **des corps finis**, généralement un corps premier **Zp** ou un corps de caractéristique 2 $\boldsymbol{F_{2^m}}$. Donc toutes les opérations sont calculées modulo P.

L'équation devient:

$$\boldsymbol{y^2 \; mod \; p = (x^3 + ax + b) \; mod \; p}$$

On note Ep(a,b) l'ensemble des couples d'entiers (x,y) qui satisfont cette équation.
On parle **de groupe elliptique**.

2.2. Arithmétique sur les courbes elliptiques

Les points $P=(x,y)$ qui satisfont l'équation de la courbe elliptique **forment un groupe sous l'addition de points**.

Le groupe des points $E(F_p)$ sur une courbe elliptique définie sur F_p **est un groupe abélien**, ce qui signifie que **<u>l'addition des points est commutative et possède une identité.</u>**

- **L'identité du groupe** est un point particulier, souvent appelé "point à l'infini", noté O.

- **L'inverse d'un point** $P=(x,y)$ est le point $P'=(x,-y)$.

2.2.1. Addition des points

L'opération fondamentale sur une courbe elliptique E est **l'addition de deux points P et Q** , cette opération va grandement dépendre du système de coordonnées considérées pour représenter un point sur E.

Les courbes sont définies **dans un corps premier Fp**, **les calculs modulaires sont donc indispensables pour limiter les coordonnées des points dans l'intervalle [0, p − 1]** ,

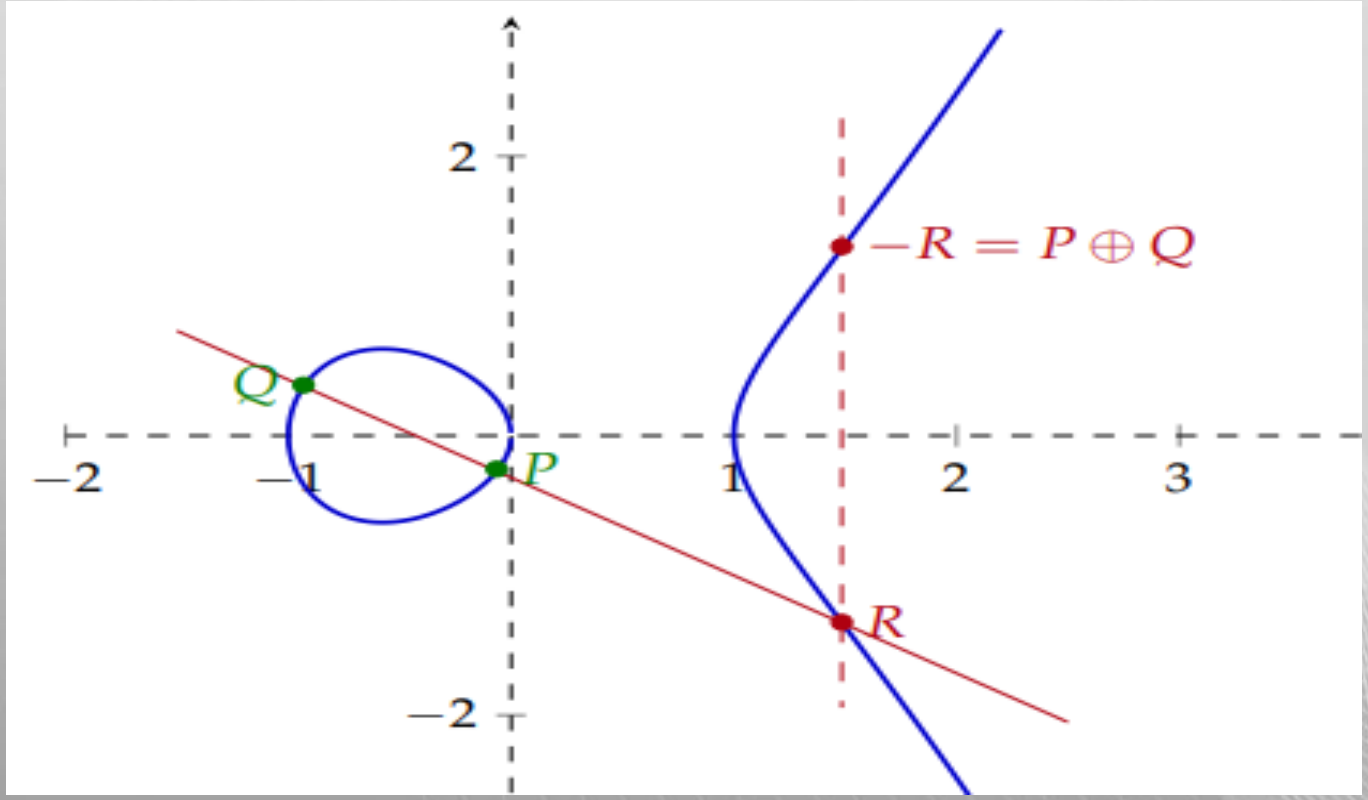
Pour P(x1, y1) , Q(x2, y2) deux points distants de la courbe E, nous définissons :

P + Q = (x3, y3), L'unité d'addition de points est utilisée par l'opérateur de multiplication scalaire .

Si Q≠ P et Q≠ −P , alors le point R = (P + Q) = (x3, y3) :

$$x_3 = (\frac{y_2 - y_1}{x_2 - x_1})^2 - x_1 - x_2 \quad (mod \quad p)$$

$$y_3 = (\frac{y_2 - y_1}{x_2 - x_1})(x_1 - x_3) - y_1 \quad (mod \quad p)$$



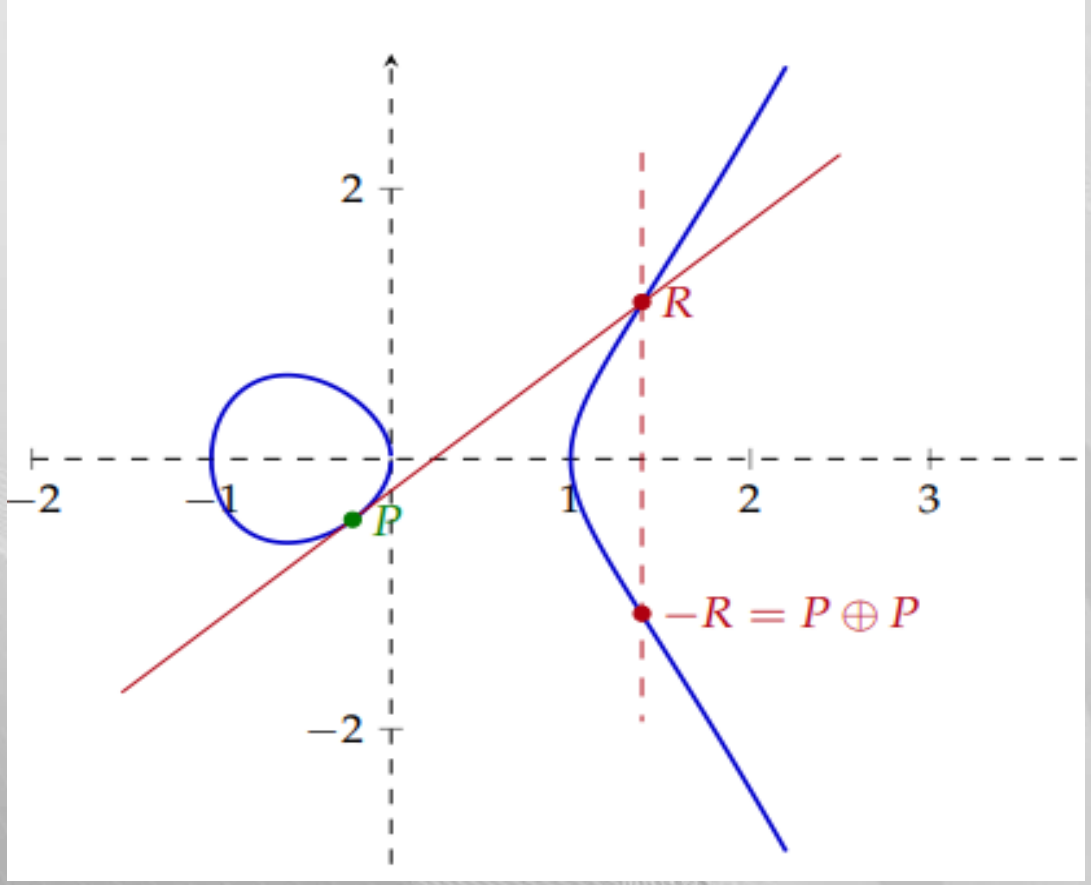
2.2.2. Doublement de points

Le calcul de doublement de point est montré dans les formules suivantes :

- Si P = Q, alors le point 2P = (x3, y3).

$$x_3 = (\frac{3x_1^2 + a}{2y_1})^2 - 2x_1 \quad (mod \quad p).$$

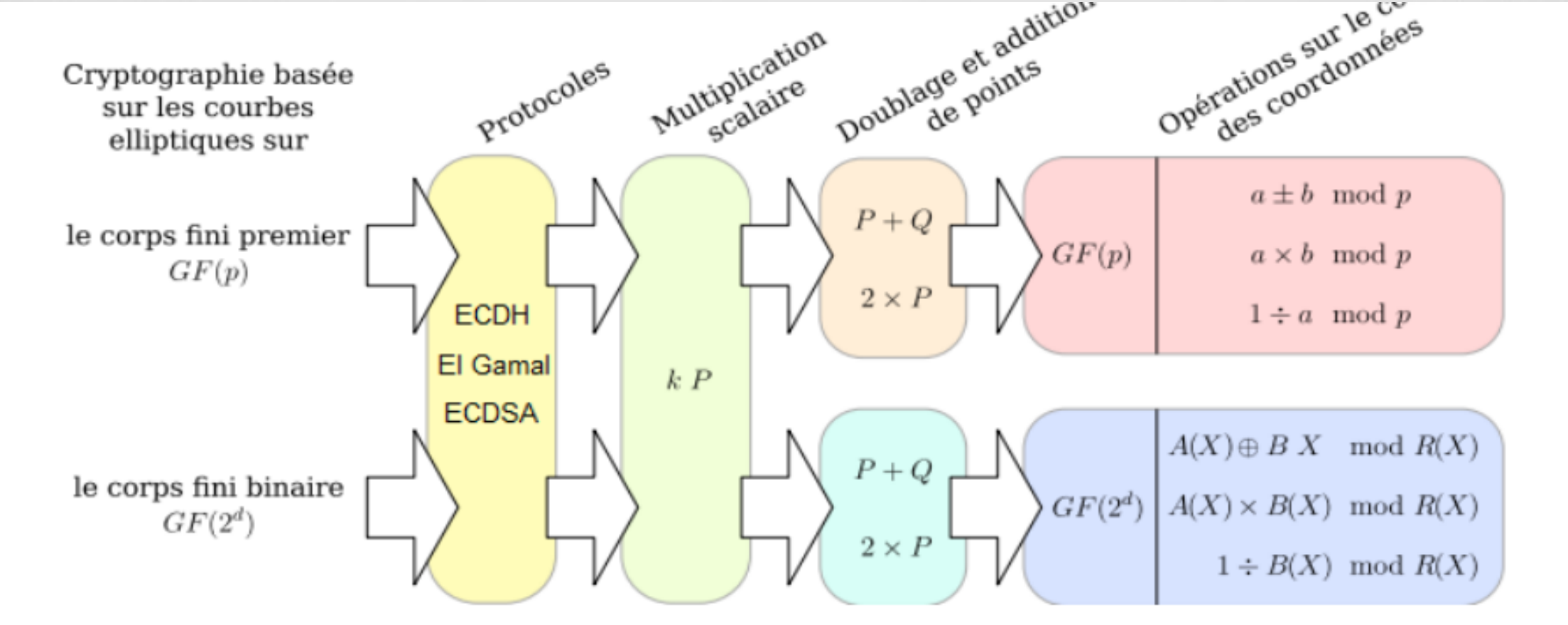
$$y_3 = (\frac{3x_1^2 + a}{2y_1})(x_1 - x_3) - y_1 \quad (mod \quad p).$$



l'addition de deux fois le point R est réalisé en traçant la tangente à la courbe en R.

2.2.3. Multiplication des points

La multiplication scalaire KP d'un point P par un entier K est obtenue en ajoutant le point P à lui-même K fois. Cette opération est fondamentale dans l'algorithme de la cryptographie à courbes elliptiques et cela est la base du **problème du logarithme discret elliptique**.


Cryptographie basée sur les courbes elliptiques sur
Protocoles
Multiplication scalaire
Doublage et addition de points
Opérations sur les coordonnées
le corps fini premier $GF(p)$
ECDH
El Gamal
ECDSA
$k\ P$
$P+Q$
$2 \times P$
$GF(p)$
$a \pm b \mod p$
$a \times b \mod p$
$1 \div a \mod p$
le corps fini binaire $GF(2^d)$
$P+Q$
$2 \times P$
$GF(2^d)$
$A(X) \oplus B\ X \mod R(X)$
$A(X) \times B(X) \mod R(X)$
$1 \div B(X) \mod R(X)$

3. Problème de logarithme discret sur les courbes elliptiques

La sécurité des systèmes basés sur les courbes elliptiques repose sur la difficulté du **problème du logarithme discret elliptique (ECDLP)**. Ce problème peut être formulé comme suit :

Soit un point **P** sur une courbe elliptique et un point **Q=kP** (où k est un entier secret), il est extrêmement difficile de déterminer **k** à partir de P et Q.

Cela rend pratiquement impossible de calculer **k** même si l'on connaît P et Q, à condition de choisir des courbes de taille suffisamment grande.

Ce problème est similaire au problème du logarithme discret classique dans les groupes multiplicatifs des entiers modulo un nombre premier, mais il est plus difficile à résoudre dans les groupes des courbes elliptiques, ce qui en fait un bon candidat pour la cryptographie.

4. Echanges des clés ECDH

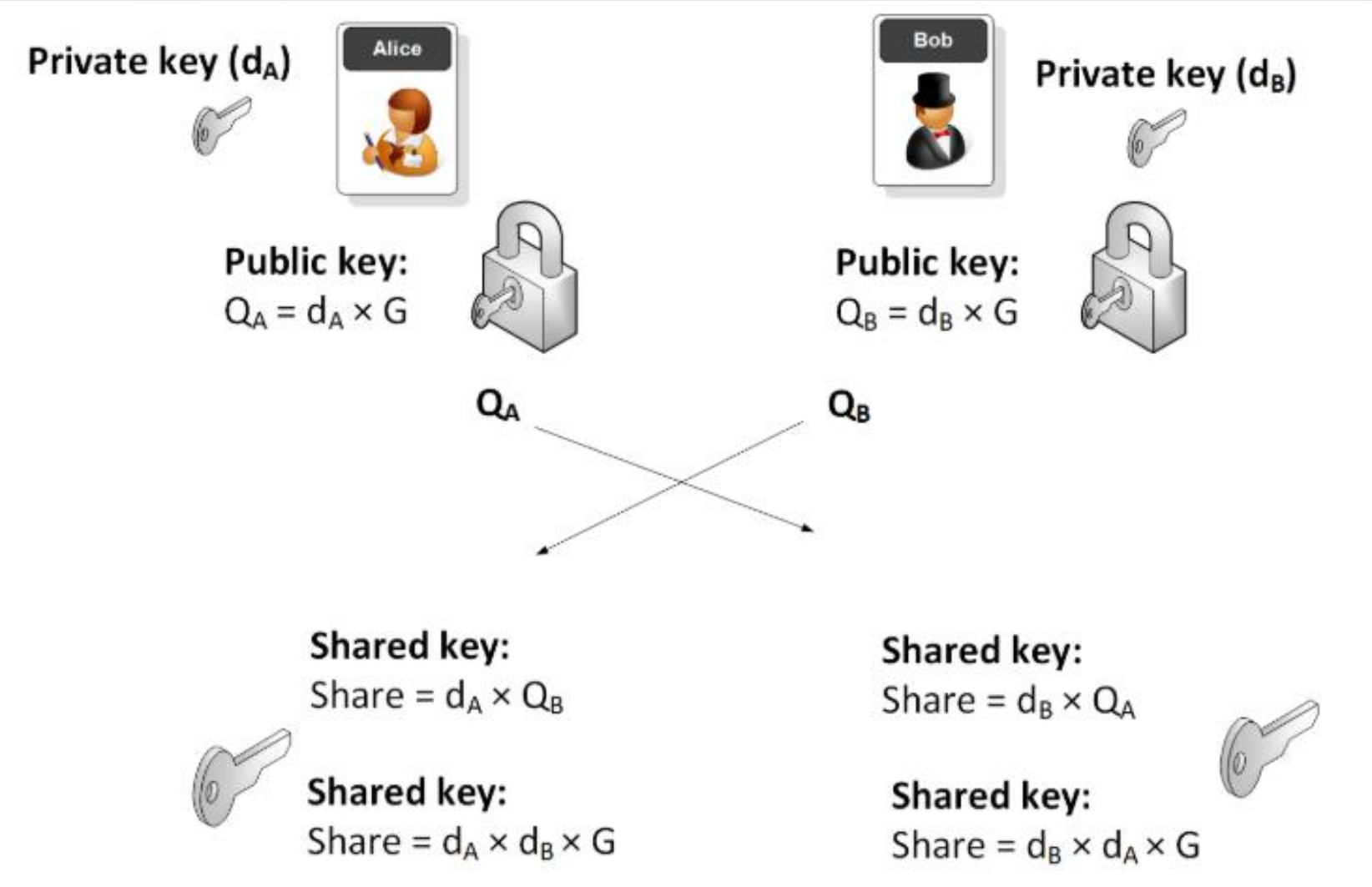
ECDH (Elliptic Curve Diffie-Hellman) est un algorithme d'échange de clés qui permet à deux parties, chacune disposant d'une paire de clés publique-privée à courbe elliptique, d'établir un **secret partagé** sur un canal non sécurisé. **L'ECDH** est très similaire à l'algorithme classique **DHKE** (Diffie–Hellman Key Exchange), mais il utilise **la multiplication des points ECC** au lieu **des exponentiations modulaires** .

Le protocole fonctionne comme suit :

- **Alice et Bob choisissent une courbe elliptique :**
$\boldsymbol{y^2 \bmod p = x^3 + ax + b \mod p}$ et le point G de la courbe (point générateur).

- Alice choisit un grand nombre premier $\boldsymbol{d_a}$ (**clé privé**) et calcule les coordonnées du point Q_a qui représente la clé publique: $Q_a = d_a\, G$.
- le même processus pour Bob $Q_b = \boldsymbol{d_b}\, G$.

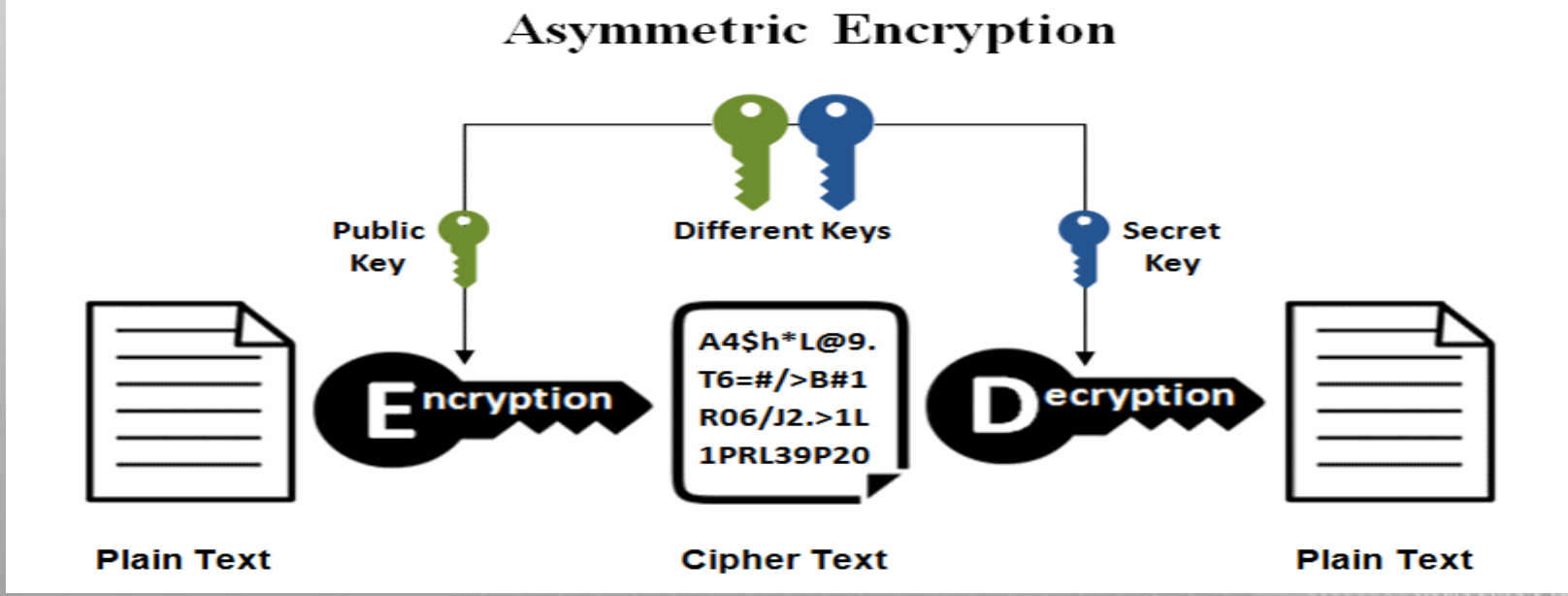
- Alice et Bob échangent des clés publiques Q_a et Q_b.


Private key (d_A)
Alice
Bob
Private key (d_B)
Public key:
$Q_A = d_A \times G$
Public key:
$Q_B = d_B \times G$
Q_A
Q_B
Shared key:
Share = $d_A \times Q_B$
Shared key:
Share = $d_B \times Q_A$
Shared key:
Share = $d_A \times d_B \times G$
Shared key:
Share = $d_B \times d_A \times G$

5. Chiffrement de ECC

Le chiffrement avec ECC (Elliptic Curve Cryptography) est largement utilisé dans la cryptographie moderne, notamment dans les systèmes de clé publique comme **ECDSA** (Elliptic Curve Digital Signature Algorithm) pour la signature et **ECDH** (Elliptic Curve Diffie-Hellman) pour l'échange de clés.

Cependant, le chiffrement pur de données avec ECC n'est pas aussi courant que les autres systèmes comme RSA, mais il est possible d'utiliser des schémas comme **ECIES** (Elliptic Curve Integrated Encryption Scheme) pour le chiffrement.



6. Schéma de chiffrement intégré à courbe elliptique (ECIES)

ECIES (Elliptic Curve Integrated Encryption Scheme) est un schéma de chiffrement hybride qui **combine la cryptographie à courbe elliptique (ECC) pour l'échange de clés et un algorithme de chiffrement symétrique pour la confidentialité des données.**

Il est basé sur les principes de l'algorithme **ECDH (Elliptic Curve Diffie-Hellman)** pour l'échange de clés, puis utilise un algorithme de chiffrement symétrique comme **AES** pour chiffrer les données avec la clé secrète partagée.

Le schéma ECIES fonctionne en trois étapes principales :

1.Échange de clés (ECDH) : Les deux parties échangent une clé secrète partagée en utilisant le mécanisme ECDH. Chaque partie a une paire de clés publiques et privées. Ils utilisent l'une des clés publiques pour générer une clé secrète partagée, sans jamais exposer leurs clés privées.

2.Génération de la clé de session (clé symétrique) : La clé secrète partagée dérivée de l'échange ECDH est ensuite **transformée en une clé symétrique (**par exemple, une clé AES) à l'aide d'une **fonction de dérivation de clé (KDF, Key Derivation Function) tellque** PBKDF2**.**

3.Chiffrement et déchiffrement des données : Une fois que la clé symétrique est dérivée, elle est utilisée pour chiffrer et déchiffrer les données avec un algorithme de chiffrement symétrique **AES (**en mode **CBC)**.

- ECIES peut aussi inclure des mécanismes d'authentification pour garantir l'intégrité des messages (par exemple, en utilisant **HMAC** ou **MAC** pour l'authentification des données).

6.1. Processus de chiffrement :

•Le message à chiffrer est d'abord haché.

•Le chiffrement est effectué en **utilisant la clé publique du destinataire** et un algorithme de chiffrement symétrique (par exemple AES) pour assurer la confidentialité.

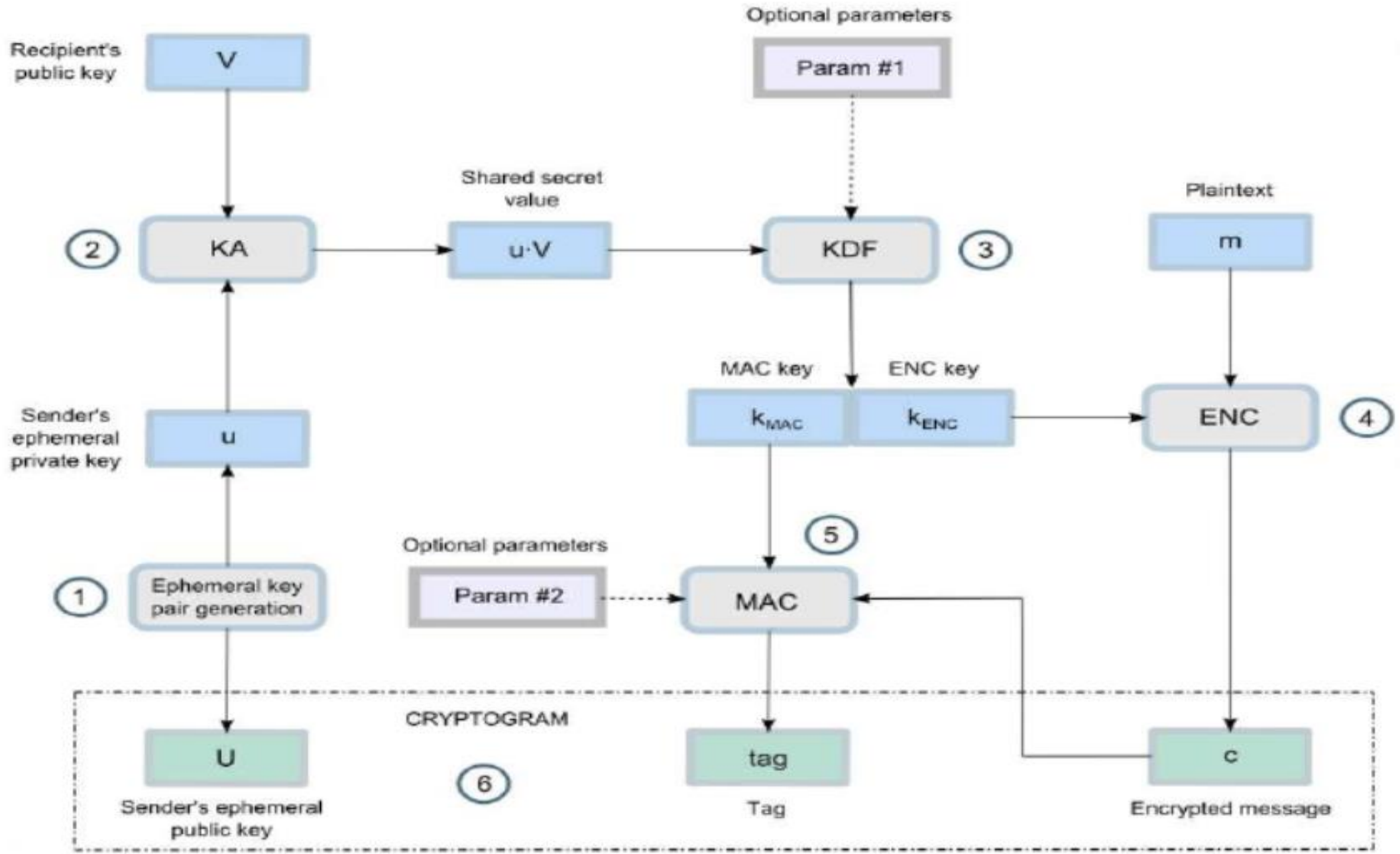
•L'émetteur génère une paire de clés ECC et envoie la clé publique au destinataire.

•La clé secrète partagée est utilisée pour générer une clé symétrique via un **KDF**.

•Cette clé symétrique est ensuite utilisée pour chiffrer le message avec un algorithme de chiffrement symétrique (comme **AES**)

•Un vecteur d'initialisation (IV) est généralement utilisé avec l'algorithme de chiffrement symétrique pour garantir la sécurité du chiffrement.

•Les données chiffrées sont envoyées au destinataire avec la clé publique utilisée, le vecteur d'initialisation (IV), et éventuellement un code d'authentification (MAC).


Optional parameters
Recipient's public key
V
Param #1
Shared secret value
Plaintext
2
KA
u·V
KDF
3
m
MAC key
ENC key
Sender's ephemeral private key
u
k_{MAC}
k_{ENC}
ENC
4
5
Optional parameters
1
Ephemeral key pair generation
Param #2
MAC
CRYPTOGRAM
U
tag
c
6
Sender's ephemeral public key
Tag
Encrypted message

6.2.Processus de Déchiffrement (par le destinataire) :

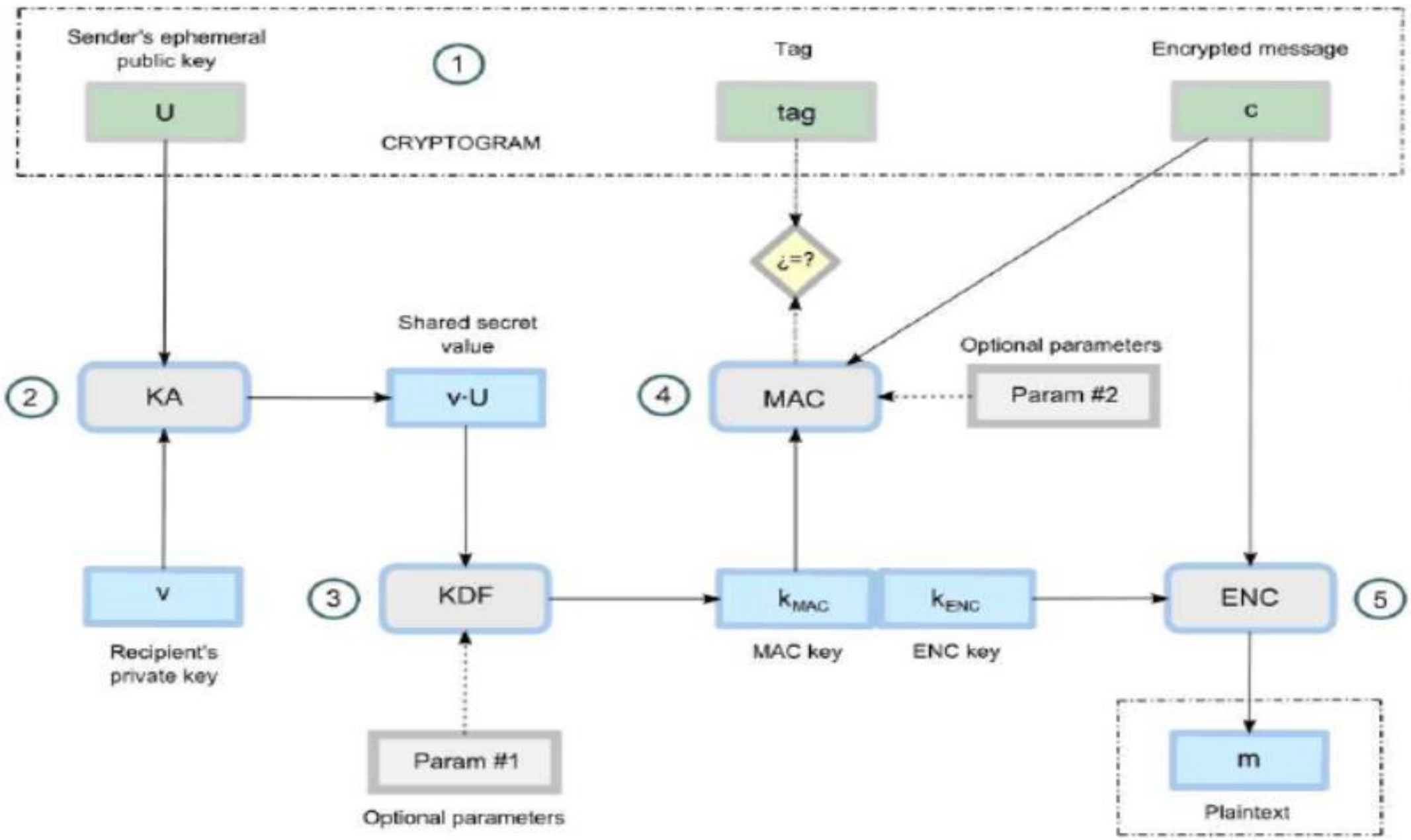
•Le destinataire reçoit les données chiffrées, la clé publique de l'émetteur, et éventuellement un code d'authentification.

•Il utilise sa propre clé privée pour dériver la clé secrète partagée via l'algorithme **ECDH** avec la clé publique de l'émetteur.

•La clé secrète partagée est utilisée pour générer la même clé symétrique via le **KDF**.

•La clé symétrique est ensuite utilisée pour déchiffrer les données avec l'algorithme de chiffrement symétrique.

•Si un code d'authentification est présent, il est vérifié pour garantir l'intégrité du message.

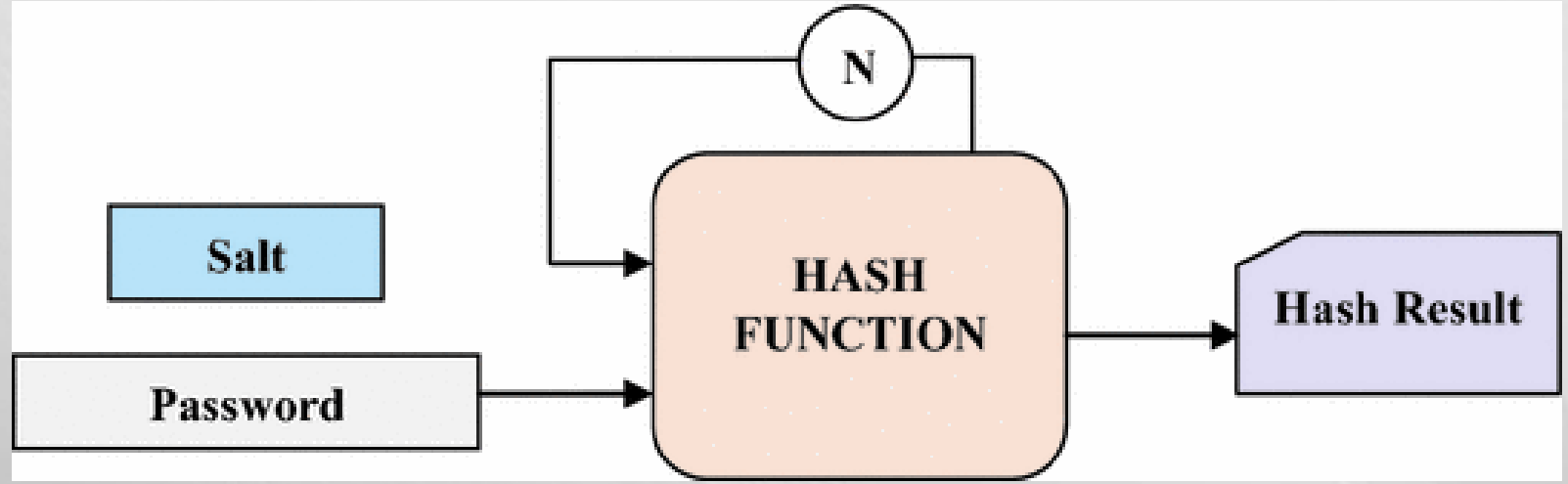

Sender's ephemeral public key
U
1
CRYPTOGRAM
Tag
tag
Encrypted message
c
¿=?
Shared secret value
Optional parameters
2
KA
v·U
4
MAC
Param #2
v
3
KDF
k_{MAC}
k_{ENC}
ENC
5
Recipient's private key
MAC key
ENC key
Param #1
Optional parameters
m
Plaintext

6.3. Fonction de dérivation de clé

Une **fonction de dérivation de clé (KDF)** est une fonction cryptographique utilisée pour générer une clé secrète à partir de données de base (telles qu'un mot de passe, une clé partagée, ou un autre type de donnée). Les KDFs sont souvent utilisés pour convertir des entrées de faible entropie (comme un mot de passe) en une clé cryptographique de taille appropriée pour les algorithmes de chiffrement modernes.

Mode de fonctionne de **KDF :**

1. **Entrée** : Une entrée est fournie, qui peut être un mot de passe, une clé partagée (comme un secret d'échange de clé), ou d'autres données.
2. **Sel (salt)** : Un sel est souvent ajouté pour garantir que la même entrée produira des clés différentes à chaque exécution (cela empêche les attaques utilisant des tables précalculées comme les attaques par table arc-en-ciel).
3. **Itérations multiples** : Un KDF applique de nombreuses itérations de hachage ou de transformations sur l'entrée et le sel. Cela augmente la complexité du processus et ralentit les attaques par force brute.
4. **Sortie** : Une clé secrète (ou une clé symétrique) est générée à la fin du processus, prête à être utilisée dans un algorithme cryptographique comme **AES** ou pour signer des données.


N
Salt
HASH
FUNCTION
Hash Result
Password

Types de KFD:

1. PBKDF2 (Password-Based Key Derivation Function 2)

PBKDF2 est un algorithme largement utilisé pour dériver des clés à partir de mots de passe ou d'autres données. Il applique des milliers d'itérations d'une fonction de hachage cryptographique, comme **SHA-256**, sur l'entrée et un **sel**.

2. Secrypt

Secrypt est Un algorithme plus moderne que PBKDF2 qui utilise de la mémoire en plus du temps pour rendre les attaques plus coûteuses.

3. HKDF (HMAC-based Key Derivation Function)

HKDF est un KDF basé sur **HMAC** (Hashed Message Authentication Code), qui est une fonction de hachage cryptographique avec une clé secrète. HKDF est souvent utilisé pour la dérivation de clés dans des systèmes de sécurité modernes, y compris dans des protocoles comme TLS, **WireGuard**, et **SSH**.

4. Argon2

Argon2 est un algorithme de dérivation de clé moderne conçu pour être sécurisé et résistant aux attaques par **ASICs** (matériel spécialisé). Il a été sélectionné comme vainqueur du concours de dérivation de clé par le mot de passe (Password Hashing Competition).

7. Crypto-système El Gamal sur des courbes elliptiques

Le **chiffrement El Gamal sur des courbes elliptiques** est une version du système de chiffrement El Gamal classique, mais qui utilise la cryptographie à courbes elliptiques (ECC) pour améliorer la sécurité et l'efficacité des clés. L'algorithme El Gamal est largement utilisé pour chiffrer des messages de manière sécurisée, et son adaptation sur des courbes elliptiques permet de bénéficier des avantages des courbes elliptiques, comme une meilleure sécurité avec des clés plus petites. C’est un protocole de chiffrement basé sur des courbes elliptiques, sur le problème du logarithme discret, il est facile de créer un analogue direct du crypto-systèmes El Gamal.

Les étapes de chiffrement EL GAMEL :

<u>Etape 1:</u>

Alice et Bob se mettent d’accord à utiliser :

- un nombre premier ***p*** particulier,
- une courbe elliptique E est choisie sur un corps fini Fp,
- **Point de base G** choisit sur la courbe elliptique E, qui est utilisé pour effectuer des multiplications de points.
- **Ordre du point G** : L'ordre n du point G est le plus petit entier tel que nG= O (le point à l'infini)

<u>Etape 2 :</u> Génération des clés

- Alice choisit Un nombre aléatoire n_A dans l'intervalle [1,n−1] qui représente la clé secrete.
- Calcule la clé publique Q_A et publie le point : $Q_A = n_A\, G$.

<u>Etape 3:</u> Chiffrement d'un message

- **Calculer** C_1 : On calcule le point C_1 qui sera utilisé pour "masquer" le message.

$$C_1 = k\, G$$

- **Calculer** C_2 : On calcule ensuite le produit C_2 , où M est le message à chiffrer: $C_2 = M + kQ_A$
- Le message chiffré est composé des deux éléments : C=(C_1, C_2)

<u>Etape 4:</u> Déchiffrement d'un message

Pour récupérer le texte en clair.

$$C_2 - n_A\, C_1 = M$$

8. Signature numérique ECDSA

L'**ECDSA** (Elliptic Curve Digital Signature Algorithm) est un algorithme de signature numérique basé sur la cryptographie à courbes elliptiques. L'objectif principal de l'ECDSA est de permettre à une entité de signer un message pour prouver :

- **L'authenticité** : Assurer que le message provient bien de la personne qui prétend l'avoir envoyé.
- **L'intégrité** : Assurer que le message n'a pas été modifié en cours de transmission.
- **Non-répudiation** : Assurer que l'expéditeur ne peut pas nier avoir signé le message.

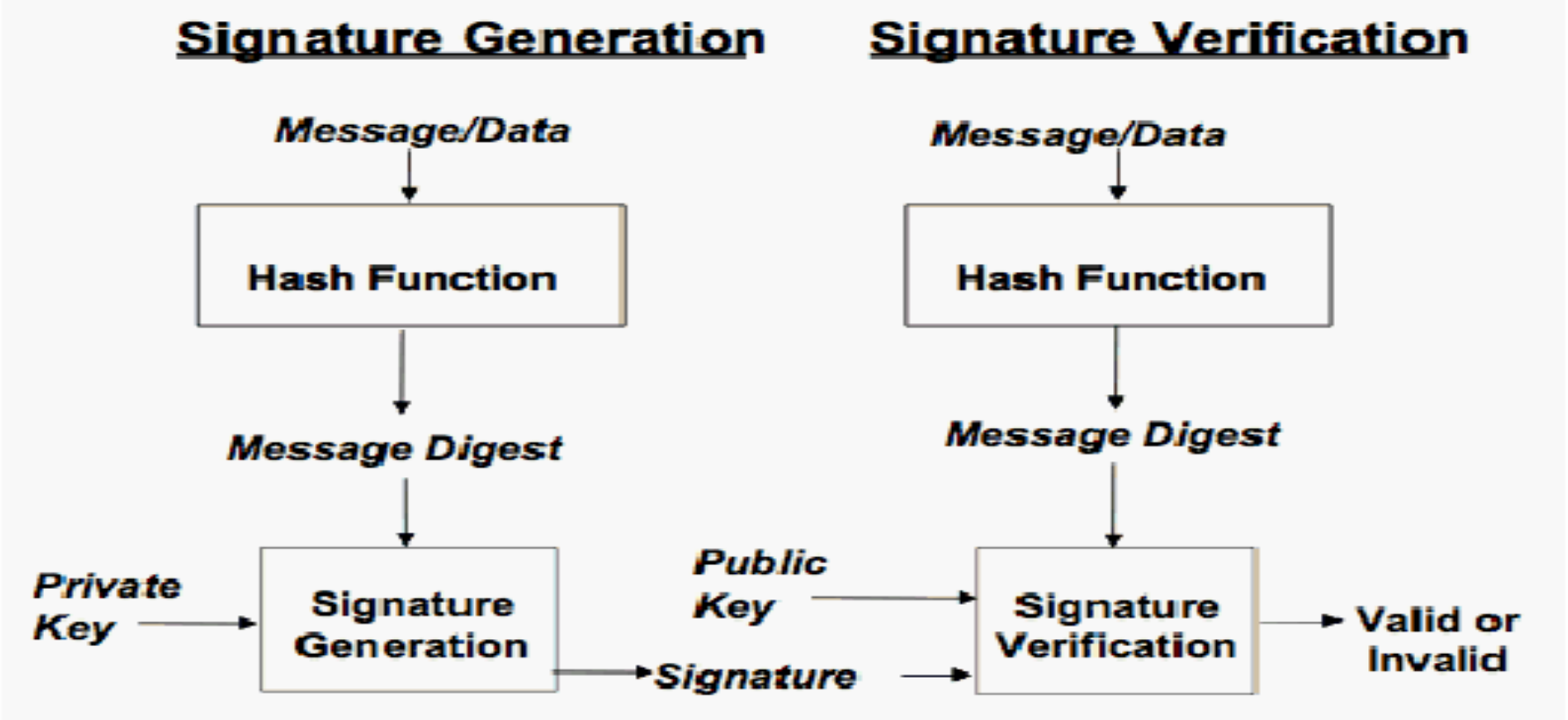
L'algorithme ECDSA est largement utilisé dans de nombreuses applications de sécurité, notamment dans les transactions de blockchain, les certificats SSL/TLS et l'authentification.

L'algorithme ECDSA repose sur trois éléments principaux :

•Une **clé privée** pour signer les messages.

•Une **clé publique** pour vérifier les signatures.

•Une **signature numérique** qui authentifie le message.



Les étapes de la signature numérique ECDSA:

<u>Etape 1:</u> Génération des clés

1. **Choisir une courbe elliptique** ainsi qu'un point de base G sur cette courbe.
2. Choisir un nombre aléatoire **d** (la clé privée) dans un intervalle [1,n-1] , où n est l'ordre du point G.
3. Calculer la clé publique Q en multipliant la clé privée d par le point G de la courbe elliptique :$Q=d\cdot G$

La clé publique Q est ensuite utilisée pour la vérification de la signature.

Etape 2: Signature du message

Supposons qu'un utilisateur souhaite signer **un message M**:

1.Calcul du hachage du message : On applique une fonction de hachage H(m) au message M pour obtenir un hachage du message.

2.Choix d'un nombre aléatoire k : Un nombre secret k est choisi de manière aléatoire dans l'intervalle [1,n−1]. Ce nombre est utilisé pour générer la signature et doit être gardé secret. Si k est utilisé plusieurs fois, la sécurité de la signature est compromise.

3.Calcul de r et s: Les deux valeurs R et S composant la signature sont calculées comme suit :

1. On calcule le point : **k G=(x1,y1),** où x1 est l'ordonnée x du point.
2. La valeur **R** est définie comme : $R = x_1 mod\ n$ Si R=0, un nouveau k est choisi.
3. On calcule la valeur **S** à partir de **la clé privée d**, **de la valeur R**, **du hachage H(M)** du message, et du **nombre aléatoire k** :

$$S = k^{-1}\ (H(M) + R.d) mod\ n$$

Où k^{-1} est l'inverse multiplicatif de k modulo n.

Si S=0, un nouveau k est choisi.

4.La signature : La signature du message M est alors constituée des deux valeurs (R,S).

<u>Etape 3:</u> Vérification de la signature

Pour vérifier une signature (R,S) pour un message M, on suit les étapes suivantes :

1.Vérification des valeurs R et S: Les valeurs R et S doivent être dans l'intervalle [1,n−1. Si elles ne sont pas dans cet intervalle, la signature est invalide.

2.Calcul du hachage du message : On applique la fonction de hachage au message M pour obtenir H(M).

3.Calcul de w : La valeur w est l'inverse de S modulo n :

w= $S^{-1} \ mod\ n$

4. Calcul des valeurs $\boldsymbol{u_1}$ $\boldsymbol{et}$ $\boldsymbol{u_2}$:

$$u_1 = H(M).w\ mod\ n$$

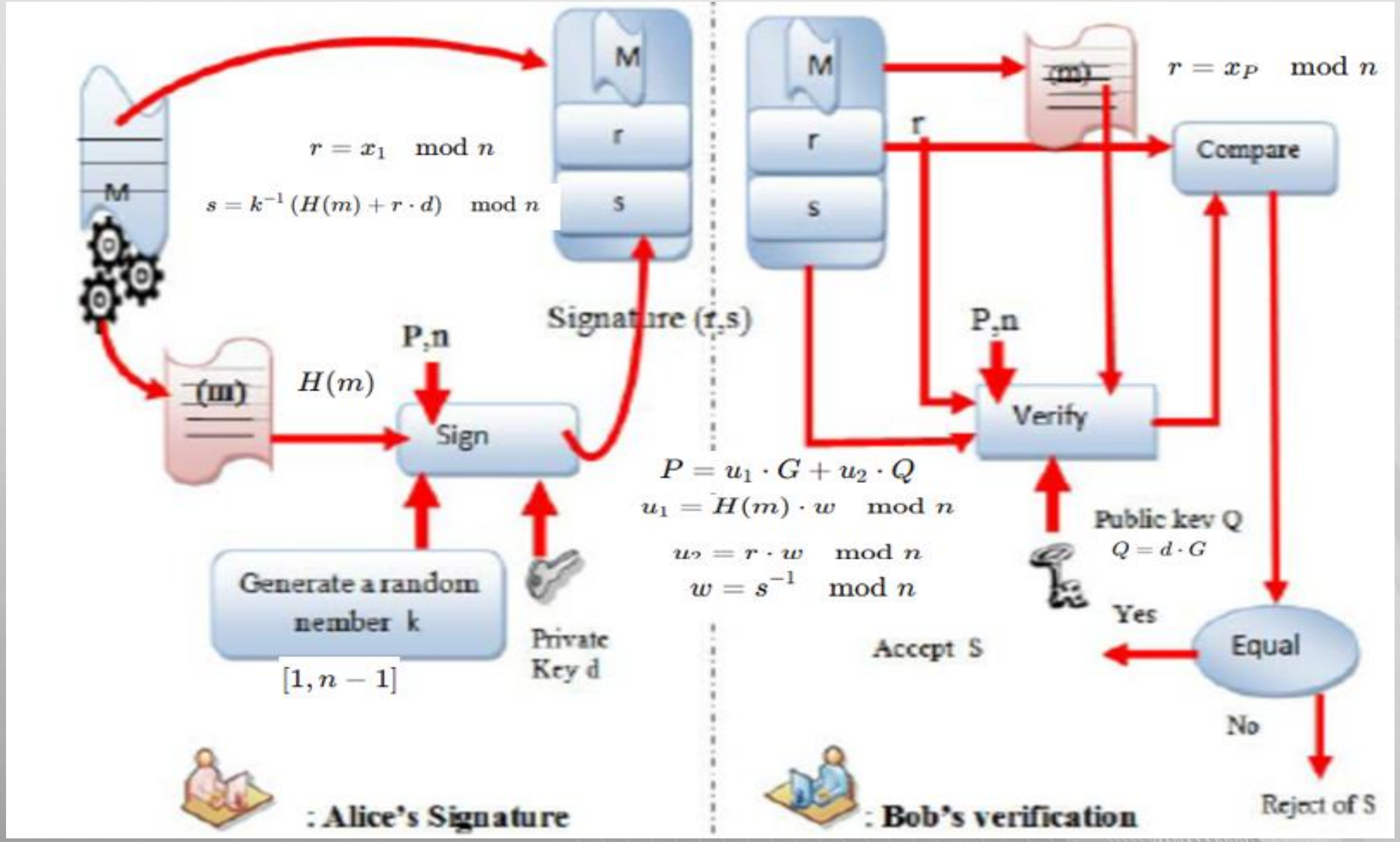
$$u_2 = R.w\ mod\ n$$

5. Calcul du point P : Le point P est calculé en combinant les valeurs $\boldsymbol{u_1}$ $\boldsymbol{et}$ $\boldsymbol{u_2}$ sur la courbe elliptique : $P = \boldsymbol{u_1}\ .\boldsymbol{G} + \ \boldsymbol{u_2}.\boldsymbol{Q}$

Cela donne un nouveau point P=(x_P,y_P)

6. Vérification de la signature : La signature est valide si et seulement si :$S = \ x_P\ mod\ n$

Si cette condition est vérifiée, la signature est valide, sinon elle est invalide.


M
r
s
$r = x_1 \mod n$
$s = k^{-1}(H(m) + r \cdot d) \mod n$
Signature (r,s)
(m)
$H(m)$
P,n
Sign
Generate a random nember k
$[1, n-1]$
Private Key d
: Alice's Signature
$r = x_P \mod n$
Compare
Verify
$P = u_1 \cdot G + u_2 \cdot Q$
$u_1 = H(m) \cdot w \mod n$
$u_2 = r \cdot w \mod n$
$w = s^{-1} \mod n$
Public key Q
$Q = d \cdot G$
Yes
Accept S
Equal
No
Reject of S
: Bob's verification

9. Domaines d’utilisation de la cryptographie à base des courbes elliptiques

De nos jours, de nombreux protocoles et plates-formes utilisent un chiffrement elliptique, en raison de sa vitesse de sa sécurité et de son efficacité, parmi ces protocoles et plateformes :

- TLS Le protocole de sécurisation d’échange sur internet, utilise la ECC dans la phase "Handshake" pour I ’agrément sur la clé à utiliser pour chaque session.

- Smart Cards à côté de RSA, les nouvelles smart-card support la ECC

- Passeport biométrique; La ECC est utilisée pour construire la clé de session.

- Bitcoin; ECC est utilisée pour la signature Transport Layer Security électronique pendant les transactions des paiements avec Bitcoin

END